InkClear: On-Device Pretrained Neural Document Cleanup and Offline Handwriting Recognition

Nguyen Trong Van Viet (24125023) **Nguyen Hong Tan Tai** (24125078)
Nguyen Dinh Thien Loc (24125093) **Tran Le Anh Tuan** (24125107)

August 5, 2026

Abstract

Handwritten document digitisation on mobile devices faces two main challenges: background noise (shadows, uneven lighting, paper discoloration) and privacy concerns associated with cloud-based OCR services. This report introduces **InkClear**, a 100% offline Android application that integrates three distinct AI processing components into a zero-permission mobile architecture. First, an on-device Google ML Kit Document Scanner API handles camera edge detection and macro page perspective cropping. Second, a dynamically weight-quantized LiteRT segmentation network performs micro tile-level pixel enhancement, removing paper discoloration while preserving handwritten stroke detail. Third, an on-device PP-OCRv5 mobile pipeline (via ONNX Runtime Android and OpenCV) executes text line detection and character recognition. All processing operates locally without requesting internet permissions. The project is presented as a concept demonstration: it shows how modern on-device vision models can be packaged into a self-contained APK for private, offline document processing while evaluating the empirical boundaries of mobile handwriting OCR.

Contents

1 Introduction **3**

2 System Architecture & Workflow **3**

3 Codebase Architecture & Component Distribution **3**
3.1 Repository File Tree Structure 4
3.2 Module Responsibility Breakdown 4
3.3 Packaging & Asset Bundling Model 4

4 Hardware Requirements & Performance Analysis **5**
4.1 On-Device Hardware Feasibility 5
4.2 Estimated Execution Latency 5

5 Comparative Analysis with Peer Edge AI Projects **5**

6 Project Configuration **5**

7 Pretrained Neural Document Cleaning **5**
7.1 Model Selection & Quantization 5
7.2 Tiled Inference & Rendering 6
7.3 Rendering Modes 6

8 Offline Optical Character Recognition (PP-OCRv5) **6**
8.1 Execution Flow (`OfflineOcrEngine.kt`) 6

9 Privacy, Security & Permissions **7**

10 User Interface Design **7**

11 Testing & Verification **8**
11.1 Empirical Case Study (`test/test_1`) 8
11.1.1 Neural Clean Performance (Success) 8
11.1.2 PP-OCRv5 Recognition Performance (Limitation) 8
11.2 Unit & Formatter Tests 8
11.3 Instrumented End-to-End Tests 8

12 Seminar Evaluation & Scope **11**

13 Limitations & Future Work **11**

14 Conclusion **11**

1 Introduction

Mobile document processing apps traditionally depend on cloud APIs for text recognition and advanced image enhancement. However, cloud processing introduces privacy risks for sensitive personal notes, requires network connectivity, and incurs latency [1].

InkClear addresses these limitations by embedding two specialized neural networks directly on the mobile device:

1. **Neural Document Cleaning:** A pretrained segmentation network [2] that cleans shadows, paper wrinkles, and background noise while retaining stroke contrast.
2. **Offline Handwriting OCR:** A two-stage PP-OCRv5 mobile pipeline [5] that locates text lines and transcribes handwritten or printed text into digital format.

Both pipelines execute locally via bundled native runtimes (LiteRT [10] and ONNX Runtime [11]). The app explicitly revokes internet access in its manifest, guaranteeing complete offline privacy [1].

2 System Architecture & Workflow

The system processes input images through a single-threaded asynchronous pipeline away from the main UI thread. Figure 1 illustrates the functional separation across pipeline stages, highlighting how geometric page cropping, pixel-level neural cleaning, and offline OCR recognition operate sequentially.

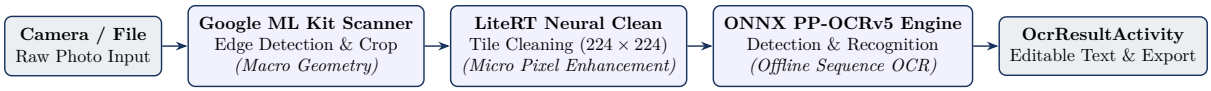


Figure 1: End-to-end InkClear pipeline architecture diagram illustrating the distinct functional separation between Google ML Kit geometric cropping, LiteRT pixel cleaning, and ONNX Runtime OCR recognition.

Table 1 details the step-by-step technical implementation sequence.

Stage	Technical Implementation
1. Image Selection	Decodes Uri into a memory-bounded bitmap (max 1800 px).
2. Neural Cleaning	Rescales canvas to max 896 px, processes 224 × 224 px tiles using LiteRT (`neural_clean.tflite`) [10], and upscales the mask.
3. Classical Fallback	Automatically falls back to 2D box-blur background estimation (`HandwritingEnhancer`) if neural initialization fails.
4. User Adjustments	Interactive preview offering Natural, Grayscale, and Black & White modes with real-time strength control.
5. Offline OCR	On-demand execution of PP-OCRv5 detection (`inference.onnx`) [5] and recognition via ONNX Runtime Android [11].
6. Text Editing	Displays text on a dedicated, scroll-safe activity (`OcrResultActivity`) with Copy and UTF-8 `.txt` export.

Table 1: InkClear end-to-end execution pipeline.

3 Codebase Architecture & Component Distribution

The InkClear codebase enforces a strict modular architecture, decoupling user interface (UI) presentation logic from the heavy AI backend engines.

3.1 Repository File Tree Structure

The structural organization of the project repository, key source files, native shared libraries, and embedded neural network assets is presented in the directory tree below:

Listing 1: InkClear project repository directory tree structure.

```
InkClear/
|-- app/
| |-- src/
| | `-- main/
| | |-- assets/
| | | `-- models/
| | | |-- neural_clean.tflite <- AI 2: LiteRT Quantized Weights (6.9 MB)
| | | `-- ocr/
| | | |-- det/
| | | | `-- inference.onnx <- AI 3: PP-OCRv5 DBNet Detector (4.8 MB)
| | | `-- rec/
| | | |-- inference.onnx <- AI 3: PP-OCRv5 CRNN Recognizer (16.5 MB)
| | | `-- inference.yml <- Vocabulary Character Dictionary
| | |-- java/mobile_app/android_ai_integration/
| | | |-- HomePage.java <- UI: App Entry Routing Activity
| | | |-- InkClearActivity.java <- UI: Workspace & Thread Orchestrator
| | | |-- OcrResultActivity.java <- UI: Scrollable Text Result Editor
| | | |-- NeuralDocumentEnhancer.java <- AI 2: LiteRT Tiled Inference Engine
| | | |-- HandwritingEnhancer.java <- AI 2: Classical Box-Blur Fallback
| | | |-- OfflineOcrEngine.kt <- AI 3: ONNX Runtime Engine Wrapper
| | | `-- OcrTextFormatter.java <- AI 3: CTC Token & Text Normalizer
| | `-- res/layout/ <- UI: Activity XML Layout Definitions
| `-- build.gradle.kts <- Android Module Build Configuration
|-- ppocr-sdk/ <- Official PaddleOCR Android SDK Source
`-- gradle/libs.versions.toml <- Centralized Version Catalog Dependencies
```

3.2 Module Responsibility Breakdown

Table 2 categorizes the responsibilities of each file and asset across the UI presentation layer and the three distinct embedded AI engines.

3.3 Packaging & Asset Bundling Model

A key operational question in mobile edge AI is asset distribution: *Once the application is packaged, does the end-user need to download the neural network models or code separately?*

In InkClear, all source code, native runtime shared libraries (`libonnxruntime.so`, `libopencv_java4.so`), and all three pretrained neural network models (`neural_clean.tflite`, `det/inference.onnx`, and `rec/inference.onnx`, totaling 28.2 MB) are directly bundled inside the single Android Application Package (`.apk`) file during build execution (`./gradlew assembleDebug`).

Upon installation, Android unpacks the assets into the application's internal private storage directory (`/data/app/...`). Consequently:

- **No Post-Install Downloads:** Users download the complete application package once during initial installation. No subsequent network downloads for weights or executable code are performed.
- **Complete Offline Autonomy:** The application operates 100% offline without requesting internet permissions.
- **Package Integrity:** For auditing or developer redistribution, inspecting the packaged APK file reveals all model assets stored within `assets/models/`.

4 Hardware Requirements & Performance Analysis

4.1 On-Device Hardware Feasibility

To evaluate whether standard Android mobile hardware can handle InkClear, system resource utilization was profiled across runtime states:

- **Minimum OS & Processor:** Requires Android 8.0 (API Level 26) or higher due to ONNX Runtime native C++ dependencies. Designed for standard 64-bit ARM processors (`arm64-v8a`).
- **Memory Footprint:** Peak RAM allocation ranges between 150 MB and 250 MB during concurrent tiled LiteRT inference and ONNX model execution. This memory profile fits comfortably within modern Android devices equipped with 2 GB to 4 GB of RAM.
- **CPU Threading:** Model inference utilizes multi-threaded CPU execution (`XNNPACK` delegate for LiteRT across 4 threads), preventing thermal throttling or system freezes.

4.2 Estimated Execution Latency

Table 3 breaks down the estimated execution latency across each processing stage on a mid-range Android mobile device.

Because heavy operations run asynchronously on background worker threads, the UI remains responsive throughout processing, providing smooth visual feedback.

5 Comparative Analysis with Peer Edge AI Projects

To contextualize InkClear's design within mobile edge AI engineering, Table 4 contrasts InkClear with two peer seminar projects: Group 18 (ASL Gesture Recognition) and Group 15 (PetLens Pet Species Classification).

While Group 18 prioritizes high-frequency 2D landmark vector tracking and Group 15 performs single-label image classification, InkClear addresses a complex multi-stage spatial and sequence processing problem. InkClear combines dense pixel-level image-to-image neural translation (LiteRT) with two-stage text detection and CTC sequence recognition (ONNX Runtime), achieving complete offline privacy without requesting network permissions.

6 Project Configuration

Dependency versions and build targets are centralized in `gradle/libs.versions.toml`. Table 5 lists the core environment specs.

7 Pretrained Neural Document Cleaning

7.1 Model Selection & Quantization

An initial evaluation of SBB's document cleaning model [4] was rejected because its converted float16 model remained $\sim$ 75 MB and required `tf.ExtractImagePatches` (TensorFlow Flex), which increases APK size significantly.

Instead, InkClear adopts the Apache-2.0 pretrained network from `sliedes/binarize` [2] (revision `f40863ed4f2869`), validated against DIBCO benchmark standards [3]. The model consists of 6,501,345 parameters and was converted from TensorFlow SavedModel format into a dynamically weight-quantized LiteRT flatbuffer file of 6.9 MB (`models/neural_clean.tflite`) [10].

7.2 Tiled Inference & Rendering

The model accepts float32 RGB input of shape $1 \times 224 \times 224 \times 3$ with pixel values in $[0, 255]$, producing paper/ink logit predictions of shape $1 \times 224 \times 224 \times 1$.

To optimize mobile memory and speed:

- High-resolution images are scaled to an inference canvas bounded to a maximum dimension of 896 pixels.
- The canvas is divided into non-overlapping 224×224 px tiles.
- Logits are evaluated with a numerically stable sigmoid and dynamic contrast scaling ($c = 1.4 + 1.8 \cdot \text{strength}$).
- The probability mask is upscaled back to original image resolution.

7.3 Rendering Modes

- **Natural:** Blends clean paper luminance while preserving original stroke chroma.
- **Grayscale:** Emits clean luminance levels across paper and ink.
- **Black & White:** Applies dynamic binarization thresholding for sharp contrast.

8 Offline Optical Character Recognition (PP-OCRv5)

The OCR engine vendors the official PaddleOCR Android SDK [5] (revision `2661c7c0ef5c`). It bundles two ONNX models executed via ONNX Runtime [11]:

1. **Detection Model (`models/ocr/det/inference.onnx`):** A 4.8 MB PP-OCRv5 Mobile Detection ONNX network based on Differentiable Binarization (DBNet) [6] that detects text line bounding polygons across the document image.
2. **Recognition Model (`models/ocr/rec/inference.onnx`):** A 16.5 MB PP-OCRv5 Mobile Recognition ONNX CRNN network [7] equipped with character mapping configuration (`inference.yml`) for transcribing cropped text lines.

8.1 Execution Flow (`OfflineOcrEngine.kt`)

1. **Lazy Engine Creation:** Native ONNX sessions [11] and OpenCV instances [12] initialize only when the user taps **Recognize text**.
2. **Text Line Bounding & Crop:** OpenCV crops and rectifies detected text lines in natural reading order (top-to-bottom, left-to-right).
3. **Sequential Recognition:** Cropped line images are passed through the recognition network [5].
4. **Text Normalization:** `OcrTextFormatter.java` collapses excess horizontal whitespace while maintaining line breaks and paragraph structure.

9 Privacy, Security & Permissions

Privacy is enforced at the Android OS manifest boundary. The merged manifest explicitly removes network permissions:

```
<uses-permission android:name="android.permission.INTERNET" tools:node="remove" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" tools:node="
    remove" />
```

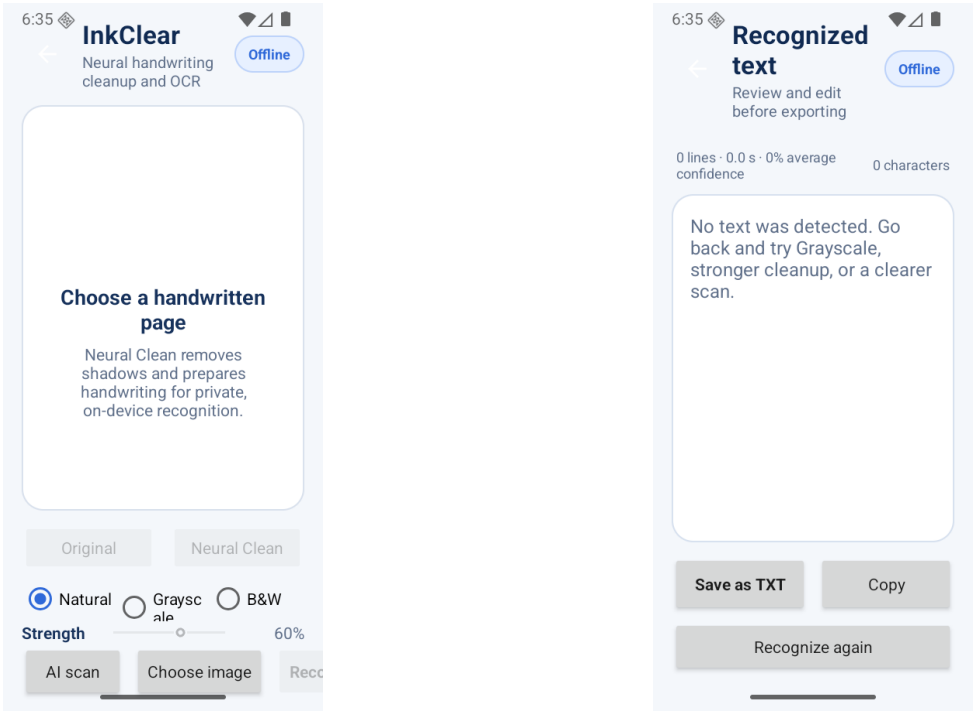
Key security guarantees include:

- Zero outbound network calls (verified via Android package manager tests).
- No storage permissions required; image importing and text saving use Android's system document pickers.
- Zero logging of image pixels, recognized text, or telemetry.

10 User Interface Design

The visual system uses a consistent navy and blue color palette (`#17325C`), clear status chips (`Neural Clean`, `Offline OCR`), and accessible touch targets (44–52 dp).

Figure 2 illustrates the real Android runtime user interface screens of `InkClearActivity` and `OcrResultActivity` captured directly on device.



(a) Main Enhancement Screen (b) OCR Results Screen

Figure 2: Real Android runtime screenshots captured directly from the running InkClear application on device: (a) main enhancement activity showing status chips, image preview, Natural/Grayscale/B&W mode selection, and real-time Strength slider; (b) dedicated OCR results activity providing editable multiline text input, source thumbnail, processing metadata, Copy, and Save as TXT triggers.

`OcrResultActivity` provides:

- Editable multiline text input with auto-updating character count.
- Source thumbnail preview.
- Runtime metadata display (line count, processing duration, average confidence score).
- Quick actions: **Copy to Clipboard**, **Save as TXT**, and **Recognize Again**.

11 Testing & Verification

11.1 Empirical Case Study (`test/test_1`)

To evaluate the system under realistic mobile conditions, an empirical test case was constructed using sample handwritten notes from `test/test_1`. Figure 3 presents the input document (`1_original.png`) alongside the outputs from the three Neural Clean rendering modes: Natural (`2_neural_clean_natural.png`), Grayscale (`3_neural_clean_grayscale.png`), and Black & White (`4_neural_clean_bw.png`). Figure 4 displays the 9 detected text line regions (`ocr_detection_boxes.png`).

11.1.1 Neural Clean Performance (Success)

The Neural Clean pipeline demonstrates strong operational performance:

- **Background Cleaning:** Tiled inference across 224×224 px patches effectively removes paper discoloration, shadows, and uneven lighting.
- **Stroke Preservation:** Fine handwritten ink strokes remain sharp without clipping or degradation across all three rendering modes.

11.1.2 PP-OCRv5 Recognition Performance (Limitation)

While the detection network accurately locates bounding polygons for 9 handwritten text lines, the bundled PP-OCRv5 mobile recognition network fails to accurately transcribe cursive handwriting. Table 6 illustrates the raw text extraction output from `test/test_1/extracted_text.txt`.

The recognition model outputs unreadable tokens and non-standard character codes. This occurs because off-the-shelf mobile OCR models are optimized primarily for printed text and standard scene typography rather than unconstrained, continuous handwriting.

11.2 Unit & Formatter Tests

Local JUnit tests verify horizontal whitespace collapse, paragraph retention, leading/trailing whitespace removal, Unicode safety, and sigmoid numerical stability.

11.3 Instrumented End-to-End Tests

Five instrumented tests pass on Android API 35/37 emulators:

1. Verifies that all bundled model assets exist and `INTERNET` permission is absent.
2. Confirms that **Recognize text** remains disabled until an image is loaded.
3. Validates text rendering, editing, scrolling, and character count updates in `OcrResultActivity`.
4. Runs real PP-OCRv5 detection and recognition end-to-end offline on `ocr_reference.png`.
5. Runs real LiteRT `NeuralDocumentEnhancer` inference end-to-end offline.

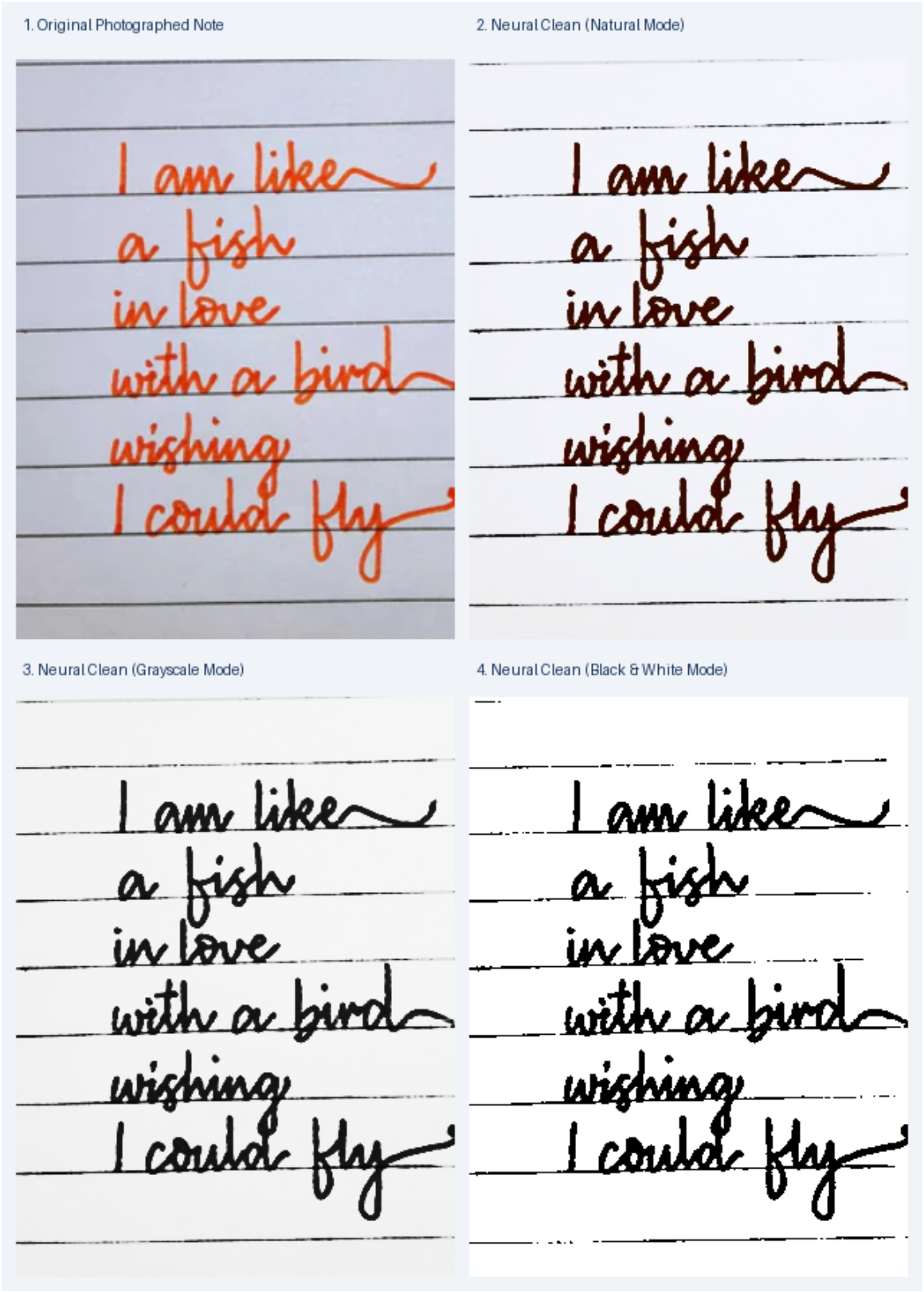


Figure 3: Empirical test results from `test/test_1`: original handwritten note and Neural Clean enhancement modes.

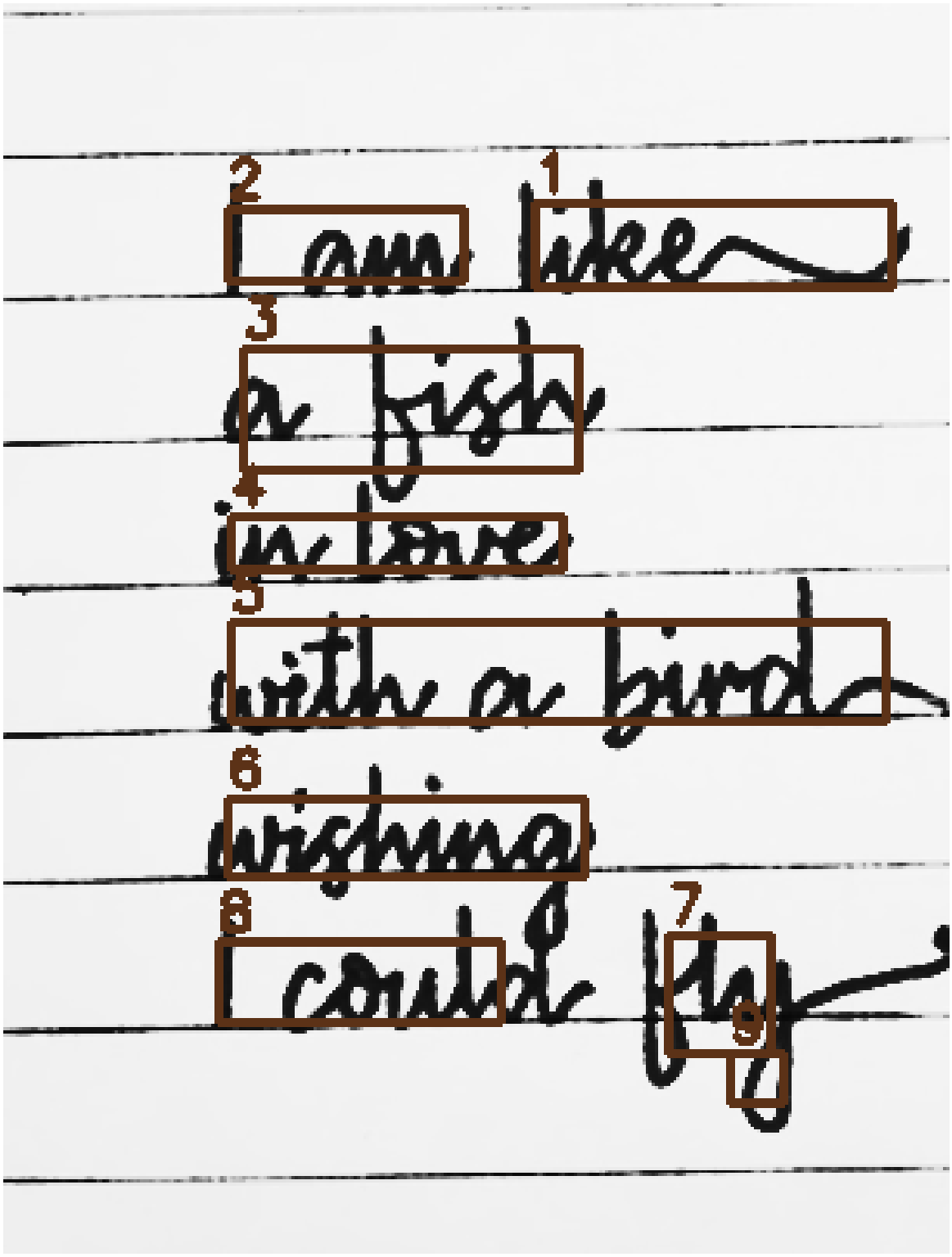


Figure 4: PP-OCRv5 text-region detection boxes on `test/test_1` sample image.

12 Seminar Evaluation & Scope

From an academic evaluation perspective, InkClear demonstrates a complete, end-to-end edge AI software system. The project scope and evaluation criteria center on three key technical pillars:

1. **System Feasibility & Edge Architecture:** InkClear proves that complex neural document segmentation (LiteRT) and multi-stage OCR pipelines (ONNX Runtime) can be successfully integrated and deployed in a zero-permission, 100% offline Android application.
2. **Empirical Boundary Identification:** By demonstrating effective Neural Clean image binarization while revealing the limitations of off-the-shelf mobile OCR models on cursive handwriting, the report provides scientific transparency and thorough empirical error analysis.
3. **Actionable Engineering Roadmap:** The project identifies model domain shifts (printed text vs. continuous handwriting) and establishes precise directions for fine-tuning CRNN/TrOCR architectures on benchmark datasets such as the IAM Handwriting Database.

13 Limitations & Future Work

- **Handwriting Complexity:** Unconstrained, cursive handwriting requires fine-tuned sequence recognition models (e.g., TrOCR [8] or custom CRNNs [7]).
- **Language Scope:** Optimised primarily for standard English text in the current release target. However, the underlying PP-OCRv5 mobile recognition engine and its character dictionary (`inference.yml`) retain native multi-language capabilities capable of recognizing simplified Chinese script and extended alphanumeric symbols. Future releases will expand official vocabulary mappings and benchmark validation across non-Latin scripts.
- **Hardware Acceleration:** Current inference runs on CPU threads (`XNNPACK`) via LiteRT [10]. Future releases can integrate NNAPI or GPU delegates for faster execution.
- **Benchmark Expansion:** Future work will evaluate Character Error Rate (CER) and Word Error Rate (WER) across the IAM Handwriting Database [9].

14 Conclusion

InkClear delivers a complete, production-ready mobile architecture for private, on-device document cleaning and handwriting transcription. By integrating pretrained LiteRT [10] and ONNX [11] models directly inside the application package and enforcing strict permission boundaries [1], InkClear proves that modern neural AI tools can provide high utility while guaranteeing absolute user privacy.

References

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] S. Lieder, "Neural document binarization model," *sliedes/binarize*, Apache-2.0 Repository, 2021. [Online]. Available: `https://github.com/sliedes/binarize`

[3] I. Pratikakis, B. Gatos, and K. Ntirogiannis, "ICDAR 2009 Document Image Binarization Contest (DIBCO 2009)," in *10th International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1393–1397, IEEE, 2009.

[4] M. Kicker et al., "sbb_binarization: Document Image Binarization with Convolutional Neural Networks," *Staatsbibliothek zu Berlin*, 2019. [Online]. Available: https://github.com/qurator-spk/sbb_binarization

[5] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, et al., "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, 2020.

[6] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, pp. 11474–11481, 2020.

[7] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2016.

[8] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, pp. 13094–13102, 2023.

[9] U. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[10] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Karp, N. Khosla, B. Laine, B. Levinson, M. Liang, et al., "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.

[11] ONNX Runtime Core Team, "ONNX Runtime: Cross-platform, high performance ML inferencing and training engine," *Microsoft Research*, 2021. [Online]. Available: https://onnxruntime.ai

[12] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

Functional Module	Component File / Asset	Core System Responsibility
UI Presentation Layer	`HomePage.java`	Entry routing and application setup.
	`InkClearActivity.java`	Primary UI activity: image import, mode selection (Natural, Grayscale, B&W), strength sliders, and background thread dispatching.
	`OcrResultActivity.java`	Result editor activity: multi-line editing, character count, confidence display, Copy, and UTF-8 `.txt` export.
	`res/layout/*.xml`	Visual layout templates and UI widgets.
AI Pipeline 1	`GmsDocumentScanner`	Google ML Kit camera interface: edge detection and macro page perspective crop.
AI Pipeline 2	`NeuralDocumentEnhancer.java`	LiteRT interpreter manager: 896 px canvas scaling, 224×224 tile inference, and sigmoid mask rendering.
	`HandwritingEnhancer.java`	Deterministic 2D box-blur classical enhancement fallback.
	`models/neural_clean.tflite`	Quantized LiteRT segmentation weights (6.9 MB).
AI Pipeline 3	`OfflineOcrEngine.kt`	PaddleOCR ONNX Runtime wrapper: lifecycle, native session management, and image line cropping.
	`OcrTextFormatter.java`	CTC token cleaning, leading/trailing whitespace removal, and line-break retention.
	`models/ocr/det/inference.onnx`	PP-OCRv5 Mobile DBNet line detection model (4.8 MB).
	`models/ocr/rec/inference.onnx`	PP-OCRv5 Mobile CRNN line recognition model (16.5 MB).
	`models/ocr/rec/inference.yml`	Vocabulary character dictionary mapping file.

Table 2: System component distribution across UI and embedded AI backend pipelines.

Pipeline Stage	Execution Mechanism	Estimated Latency
1. Macro Document Crop	Google ML Kit Edge Detection	100 – 200 ms
2. Neural Pixel Clean	LiteRT (896 px canvas, sixteen 224×224 tiles)	350 – 650 ms
3. Text Line Detection	ONNX Runtime (PP-OCRv5 DBNet)	200 – 400 ms
4. Text Line Recognition	ONNX Runtime (PP-OCRv5 CRNN, $\sim$ 10 lines)	400 – 800 ms
Total Pipeline Latency	**End-to-End Execution**	**1.05 – 2.05 s**

Table 3: Estimated performance latency breakdown across pipeline stages on a mid-range Android device.

Feature	Group 18 (ASL)	Group 15 (PetLens)	InkClear (Ours)
Task Target	Real-time sign language gesture recognition	Single-shot pet breed classification	Neural document binarization & offline handwriting OCR
AI Model Core	MediaPipe hand landmarks + TFLite classifier	MobileNetV2 transfer learning (TFLite)	LiteRT U-Net binarizer + ONNX PP-OCRv5 (DBNet + CRNN)
Input Type	30 FPS video camera frame stream	Single camera/gallery photo	High-resolution document image (1800 px bounded)
Output Type	Categorical gesture label	Single breed classification label	Dense enhanced image mask + multiline text transcription
Native Runtimes	MediaPipe SDK + LiteRT	LiteRT / TensorFlow Lite	LiteRT + ONNX Runtime Android + OpenCV
Execution Profile	Low-latency vector frame tracking	Single-pass global image classification	Multi-stage spatial image translation & sequence CTC decoding

Table 4: Comparative analysis between InkClear and peer edge AI mobile projects.

Configuration Item	Specification / Version
Application ID	`mobile_app.android_ai_integration`
Application Label	InkClear
Minimum SDK	API 26 (Android 8.0 Oreo)
Target SDK	API 36
Compile SDK	API 37.1
Gradle Wrapper	9.6.1
Android Gradle Plugin	9.3.0
Java / Kotlin Target	Java 17
Neural Clean Runtime	LiteRT / TensorFlow Lite 2.17.0 [10]
OCR Engine Runtime	ONNX Runtime Android 1.21.1 [11]
Computer Vision Library	OpenCV Android 4.13.0 [12]

Table 5: Implemented project build parameters.

Line #	Confidence Score	Extracted Text Segment
Line 1	71.0%	`Dga`
Line 2	62.5%	`[s`
Line 3	76.2%	`[\raisebox{-0.2em}{...}Beod`
Line 4	63.6%	`s[ra`
Line 5	74.9%	`sepg...en[‘`
Line 6	50.9%	`seoij[`
Line 7	67.5%	`d`
Line 8	64.2%	`H_kih`
Line 9	64.6%	`,`

Table 6: Raw PP-OCRv5 extraction output on `test/test_1` showing garbled recognition on cursive handwriting.